



Universidade Estadual de Santa Cruz – UESC

Relatório de Implementações de Métodos da Disciplina Análise Numérica

**Relatório de implementações
realizadas por Davi Roriz Oliveira**

Disciplina: Análise Numérica

Curso: Ciência da Computação

Semestre 2024.1

Professor: Gesil Sampaio Amarante II

**Ilhéus – BA
2024**

Índice

1. Lista de figuras.....	2
2. Linguagem escolhida e justificativas.....	3
3. Método da bissecção.....	4
4. Método da falsa posição.....	6
5. Método de Newton-Raphson.....	8
6. Método da secante.....	10
7. Método da eliminação de Gauss.....	12
8. Método da fatoração LU.....	14
9. Método de Jacobi.....	16
10. Método de Gauss-Seidel.....	18
11. Matriz inversa.....	20
12. Condicionamento de matriz.....	22
13. Considerações finais.....	24

1. Lista de Figuras

Não foram utilizadas figuras no relatório.

2. Linguagem escolhida e justificativas

Para a realização do projeto, foi utilizado a linguagem de programação Python na sua versão 3.11.2 que é uma linguagem de alto nível e que possui uma facilidade de escrita e leitura de código. Sua sintaxe é simples comparada a outras linguagens, possibilitando que programadores de diferentes níveis possam aprender e manipular códigos em Python.

Sua escolha se deu ao fato de sua ampla variedade de bibliotecas especializadas para realizar algoritmos matemáticos, como Math, Os e Numpy, visto que elas possuem uma variedade de funções que não se encontram em outras linguagens. Além de serem de fácil instalação, elas também são multiplataforma, podendo operar em diferentes sistemas operacionais, bem como Windows, Linux e MacOS.

3. Método da bissecção

Estratégia de Implementação:

O arquivo contém duas funções principais: A função ‘bissec’ que realiza o cálculo do método e tem como parâmetros a função ‘f’, os valores ‘a’ e ‘b’, e o valor da tolerância ‘tol’. Dentro da função é calculado a quantidade de iterações que vão ocorrer para cada teste, a condição de bolzano para verificar se os sinais de $f(a)$ e $f(b)$ são opostos, e o cálculo do método em si.

A outra função, é a main, nela foi utilizada a biblioteca do Python, Os, necessária para poder ser encontrado o diretório em que o arquivo se encontra e identificar os arquivos ‘in.txt’ e ‘out.txt’, que são os arquivos de entrada e saída respectivamente. Após isso, ele abre o arquivo de entrada e puxa os dados que há nele, e então abre o arquivo de saída e para cada caso teste (foi utilizado um laço for para poder realizar mais de um para cada execução) é feita uma chamada da função ‘bissec’ para calcular os resultados. Por fim, o resultado final é escrito no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

O arquivo de entrada consiste em 4 linhas: a função, o valor de ‘a’, o valor de ‘b’, e a tolerância, também é possível por mais de um caso teste por arquivo, então o ‘in.txt’ que foi enviado possui 5 casos teste: dois referentes ao exercício 3.3, um do exercício 3.6, e dois do exercício 3.8 das páginas 97-100 do livro Cálculo Numérico Neide Franco.

O arquivo de saída consiste em uma única linha, contendo o resultado do problema teste.

Problema teste:

1. Problema 3.3:

O problema consiste em testar a função $g(x)$ para $g = 0.1$ e $g = 0.9$. A equação original do problema tinha a variável ‘T’, que foi substituída por ‘x’ para maior facilidade de entendimento. A equação foi rearranjada para $g - (1 + x + x^2/2)e^{-x}$.

Para $g = 0.1$, os valores de 'a' e 'b' foram 5 e 6, respectivamente, esses valores foram escolhidos após a equação ser posta no programa web GeoGebra, que gerou o gráfico da função, então foi visto que a raiz da função se encontrava entre esses valores inteiros. O resultado encontrado foi 5,3223114013671875.

Para $g = 0.9$, os valores de 'a' e 'b' foram 1 e 2, respectivamente, esses valores foram escolhidos após a equação ser posta no programa web GeoGebra, que gerou o gráfico da função, então foi visto que a raiz da função se encontrava entre esses valores inteiros. O resultado encontrado foi 1,1020660400390625.

2. Problema 3.6:

Neste problema, deve-se calcular a equação: $\text{tg}(\theta/2) = \frac{\text{sen}(\alpha) \cdot \cos(\alpha)}{g \cdot R/v^2 - \cos^2(\alpha)}$. E os valores de 'a' e 'b' foram 1 e 2, respectivamente. O teste foi realizado e o valor obtido foi de: 1,0237579345703125.

3. Problema 3.8:

Este problema, assim como o 3.3, também apresenta dois cenários com valores diferentes para as variáveis "k" e "p". A equação final é kx , e as instruções para obtenção dos valores de "k" e "p" podem ser encontradas no livro. O teste foi realizado para $k = 6.1$ e $p = 9$, e para $k = 7.5$ e $p = 12$. Os valores de 'a' e 'b' foram 1 e 2, respectivamente. O valor obtido foi o mesmo para ambos os casos: 1,9999847412109375.

Dificuldades enfrentadas:

Como dito anteriormente, para as escolhas dos valores de 'a' e 'b', foi necessário pôr as equações no programa web GeoGebra, e analisar os gráficos das equações e identificar quais os valores inteiros que as raízes dos problemas se encontravam.

4. Método da falsa posição

Estratégia de Implementação:

O arquivo contém duas funções principais: A função ‘falsa_pos’ que realiza o cálculo do método e tem como parâmetros a função ‘f’, os valores ‘a’ e ‘b’, e o valor da tolerância ‘tol’. Dentro da função é calculado a quantidade de iterações que vão ocorrer para cada teste, a condição de bolzano para verificar se os sinais de $f(a)$ e $f(b)$ são opostos, e o cálculo do método em si.

A outra função, é a main, nela foi utilizada a biblioteca do Python, Os, necessária para poder ser encontrado o diretório em que o arquivo se encontra e identificar os arquivos ‘in.txt’ e ‘out.txt’, que são os arquivos de entrada e saída respectivamente. Após isso, ele abre o arquivo de entrada e puxa os dados que há nele, e então abre o arquivo de saída e para cada caso teste (foi utilizado um laço for para poder realizar mais de um para cada execução) é feita uma chamada da função ‘falsa_pos’ para calcular os resultados. Por fim, o resultado final é escrito no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

O arquivo de entrada consiste em 4 linhas: a função, o valor de ‘a’, o valor de ‘b’, e a tolerância, também é possível por mais de um caso teste por arquivo, então o ‘in.txt’ que foi enviado possui 5 casos teste: dois referentes ao exercício 3.3, um do exercício 3.6, e dois do exercício 3.8 das páginas 97-100 do livro Cálculo Numérico Neide Franco.

O arquivo de saída consiste em uma única linha, contendo o resultado do problema teste.

Problema teste:

1. Problema 3.3:

O problema consiste em testar a função $g(x)$ para $g = 0.1$ e $g = 0.9$. A equação original do problema tinha a variável ‘T’, que foi substituída por ‘x’ para maior facilidade de entendimento. A equação foi rearranjada para $g - (1 + x + x^2/2)e^{-x}$.

Para $g = 0.1$, os valores de 'a' e 'b' foram 5 e 6, respectivamente. O resultado encontrado foi 5,161160168917105.

Para $g = 0.9$, os valores de 'a' e 'b' foram 1 e 2, respectivamente. O resultado encontrado foi 1,5510326641238699.

2. Problema 3.6:

Neste problema, deve-se calcular a equação: $\text{tg}(\theta/2) = \frac{\text{sen}(\alpha) \cdot \cos(\alpha)}{g \cdot R/v^2 - \cos^2(\alpha)}$. E os valores de 'a' e 'b' foram 1 e 2, respectivamente. O teste foi realizado e o valor obtido foi de: 1,0237621399089831.

3. Problema 3.8:

Este problema, assim como o 3.3, também apresenta dois cenários com valores diferentes para as variáveis "k" e "p". A equação final é kx , e as instruções para obtenção dos valores de "k" e "p" podem ser encontradas no livro. O teste foi realizado para $k = 6.1$ e $p = 9$, e para $k = 7.5$ e $p = 12$. Os valores de 'a' e 'b' foram 1 e 2, respectivamente. O valor obtido foi o mesmo para ambos os casos: 1,5.

Dificuldades enfrentadas:

Assim como relatado no método da bissecção, para as escolhas dos valores de 'a' e 'b', foi necessário pôr as equações no programa web GeoGebra, e analisar os gráficos das equações e identificar quais os valores inteiros que as raízes dos problemas se encontravam.

5. Método de Newton-Raphson

Estratégia de Implementação:

O arquivo contém duas funções principais: A função ‘newton_raphson’ que realiza o cálculo do método e tem como parâmetros a função ‘f’, o valor de ‘x0’, e o valor da tolerância ‘tol’. Dentro da função foi feita uma função para calcular a derivada de $f(x)$, e o cálculo do método em si, que consta um total de 1000 iterações, sendo retornado um erro caso não haja a convergência após essas iterações.

A outra função, é a main, nela foi utilizada a biblioteca do Python, Os, necessária para poder ser encontrado o diretório em que o arquivo se encontra e identificar os arquivos ‘in.txt’ e ‘out.txt’, que são os arquivos de entrada e saída respectivamente. Após isso, ele abre o arquivo de entrada e puxa os dados que há nele, e então abre o arquivo de saída e para cada caso teste (foi utilizado um laço for para poder realizar mais de um para cada execução) é feita uma chamada da função ‘newton_raphson’ para calcular os resultados. Por fim, o resultado final é escrito no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

O arquivo de entrada consiste em 3 linhas: a função, o valor de ‘x0’, e a tolerância, também é possível por mais de um caso teste por arquivo, então o ‘in.txt’ que foi enviado possui 5 casos teste: dois referentes ao exercício 3.3, um do exercício 3.6, e dois do exercício 3.8 das páginas 97-100 do livro Cálculo Numérico Neide Franco.

O arquivo de saída consiste em uma única linha, contendo o resultado do problema teste.

Problema teste:

1. Problema 3.3:

O problema consiste em testar a função $g(x)$ para $g = 0.1$ e $g = 0.9$. A equação original do problema tinha a variável ‘T’, que foi substituída por ‘x’ para maior facilidade de entendimento. A equação foi rearranjada para $g - (1 + x + x^2/2)e^{-x}$.

Para $g = 0.1$, o valor de 'x0' foi 5. O resultado encontrado foi 5.32232033783421.

Para $g = 0.9$, o valor de 'x0' foi 1. O resultado encontrado foi 1.102065328249321.

2. Problema 3.6:

Neste problema, deve-se calcular a equação: $\text{tg}(\theta/2) = \frac{\text{sen}(\alpha) \cdot \cos(\alpha)}{g \cdot R/v^2 - \cos^2(\alpha)}$. O valor de 'x0' foi 1. O teste foi realizado e o valor obtido foi de: 1.0237621399089833.

3. Problema 3.8:

Este problema, assim como o 3.3, também apresenta dois cenários com valores diferentes para as variáveis "k" e "p". A equação final é kx , e as instruções para obtenção dos valores de "k" e "p" podem ser encontradas no livro. O teste foi realizado para $k = 6.1$ e $p = 9$, e para $k = 7.5$ e $p = 12$. O valor de 'x0' foi 1. O valor obtido foi o mesmo para ambos os casos: 1.

Dificuldades enfrentadas:

Não houve dificuldades na implementação desse método.

6. Método da secante

Estratégia de Implementação:

O arquivo contém duas funções principais: A função ‘secante’ que realiza o cálculo do método e tem como parâmetros a função ‘f’, os valores ‘x0’ e ‘x1’, e o valor da tolerância ‘tol’, além de ser feita um total de 1000 iterações para realizar o método e, caso não haja a convergência após isso, a função retorna um erro.

A outra função, é a main, nela foi utilizada a biblioteca do Python, Os, necessária para poder ser encontrado o diretório em que o arquivo se encontra e identificar os arquivos ‘in.txt’ e ‘out.txt’, que são os arquivos de entrada e saída respectivamente. Após isso, ele abre o arquivo de entrada e puxa os dados que há nele, e então abre o arquivo de saída e para cada caso teste (foi utilizado um laço for para poder realizar mais de um para cada execução) é feita uma chamada da função ‘secante’ para calcular os resultados. Por fim, o resultado final é escrito no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

O arquivo de entrada consiste em 4 linhas: a função, o valor de ‘x0’, o valor de ‘x1’, e a tolerância, também é possível por mais de um caso teste por arquivo, então o ‘in.txt’ que foi enviado possui 5 casos teste: dois referentes ao exercício 3.3, um do exercício 3.6, e dois do exercício 3.8 das páginas 97-100 do livro Cálculo Numérico Neide Franco.

O arquivo de saída consiste em uma única linha, contendo o resultado do problema teste.

Problema teste:

1. Problema 3.3:

O problema consiste em testar a função $g(x)$ para $g = 0.1$ e $g = 0.9$. A equação original do problema tinha a variável ‘T’, que foi substituída por ‘x’ para maior facilidade de entendimento. A equação foi rearranjada para $g - (1 + x + x^2/2)e^{-x}$.

Para $g = 0.1$, os valores de 'x0' e 'x1' foram 5 e 6, respectivamente. O resultado encontrado foi 5,32232033783421.

Para $g = 0.9$, os valores de 'x0' e 'x1' foram 1 e 2, respectivamente. O resultado encontrado foi 1.1020653282484845.

2. Problema 3.6:

Neste problema, deve-se calcular a equação: $\text{tg}(\theta/2) = \frac{\text{sen}(\alpha) \cdot \cos(\alpha)}{g \cdot R/v^2 - \cos^2(\alpha)}$. E os valores de 'x0' e 'x1' foram 1 e 2, respectivamente. O teste foi realizado e o valor obtido foi de: 1.023762139908972.

3. Problema 3.8:

Este problema, assim como o 3.3, também apresenta dois cenários com valores diferentes para as variáveis "k" e "p". A equação final é kx , e as instruções para obtenção dos valores de "k" e "p" podem ser encontradas no livro. O teste foi realizado para $k = 6.1$ e $p = 9$, e para $k = 7.5$ e $p = 12$. Os valores de 'x0' e 'x1' foram 1 e 2, respectivamente. O valor obtido foi o mesmo para ambos os casos: 1.

Dificuldades enfrentadas:

Não houve dificuldades na implementação desse método.

7. Método da eliminação de Gauss

Estratégia de Implementação:

O arquivo contém três funções principais: A função ‘gauss’, que tem como parâmetros a matriz ‘A’ e a matriz ‘B’, e que realiza o cálculo do método, além de chamar a função ‘substituicoes_somatoria’ se o determinante for diferente de zero, caso contrário, ele retorna um erro de coeficientes singulares da matriz.

A outra função, é a main, nela foi utilizada a biblioteca do Python, Os, necessária para poder ser encontrado o diretório em que o arquivo se encontra e identificar os arquivos ‘in.txt’ e ‘out.txt’, que são os arquivos de entrada e saída respectivamente. Após isso, ele abre o arquivo de entrada e puxa os dados que há nele, e então abre o arquivo de saída e para cada caso teste (foi utilizado um laço for para poder realizar mais de um para cada execução) é feita uma chamada da função ‘gauss’ para calcular os resultados. Por fim, o resultado final é escrito no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

O arquivo de entrada consiste em N linhas da matriz ‘A’ e uma linha da matriz ‘B’, também é possível por mais de um caso teste por arquivo, então o ‘in.txt’ que foi enviado possui 2 casos testes, o exercício 4.1 e o exercício 4.6 das páginas 150-153 do livro Cálculo Numérico Neide Franco.

O arquivo de saída consiste em N linhas, contendo os resultados do problema teste.

Problema teste:

1. Problema 4.1

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{bmatrix} 8 & -4 & -2 \\ -4 & 6 & -2 \\ -2 & -2 & 10 \end{bmatrix}$$

$$B = \begin{bmatrix} 10 \\ 0 \\ 4 \end{bmatrix}$$

E teve como resultados obtidos:

$$x[0] = 2.7586206896551726$$

$$x[1] = 2.310344827586207$$

$$x[2] = 1.4137931034482758$$

2. Problema 4.6

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{bmatrix} 1 & 4 & 4 \\ 4 & 7 & 9 \\ 4 & 7 & 18 \end{bmatrix}$$

$$B = \begin{bmatrix} 100 & 100 & 100 \\ 100 & 100 & 100 \\ 100 & 100 & 100 \end{bmatrix}$$

E teve como resultados obtidos:

$$x[0] = 3.658536585365854$$

$$x[1] = 12.195121951219512$$

$$x[2] = 0.0$$

Dificuldades enfrentadas:

Não houve dificuldades na implementação desse método.

8. Método da fatoração LU

Estratégia de Implementação:

O arquivo contém cinco funções principais: A função ‘`resolve_sistema`’, que tem como objetivo chamar as demais funções, ‘`fatoracao_LU`’ que faz o calculo principal, e ‘`substituicao_regressiva`’ e ‘`substituicao_progressiva`’ que calculam o sistema linear triangular inferior e superior respectivamente.

A outra função, é a main, nela foi utilizada a biblioteca do Python, `Os`, necessária para poder ser encontrado o diretório em que o arquivo se encontra e identificar os arquivos ‘`in.txt`’ e ‘`out.txt`’, que são os arquivos de entrada e saída respectivamente. Após isso, ele abre o arquivo de entrada e puxa os dados que há nele, e então abre o arquivo de saída e para cada caso teste (foi utilizado um laço for para poder realizar mais de um para cada execução) é feita uma chamada da função ‘`resolve_sistema`’ para calcular os resultados. Por fim, o resultado final é escrito no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

O arquivo de entrada consiste em N linhas da matriz ‘A’ e uma linha da matriz ‘B’, também é possível por mais de um caso teste por arquivo, então o ‘`in.txt`’ que foi enviado possui 2 casos testes, o exercício 4.1 e o exercício 4.6 das páginas 150-153 do livro Cálculo Numérico Neide Franco.

O arquivo de saída consiste em N linhas, contendo os resultados do problema teste.

Problema teste:

3. Problema 4.1

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{bmatrix} 8 & -4 & -2 \\ -4 & 6 & -2 \\ -2 & -2 & 10 \end{bmatrix}$$

$$B = \begin{bmatrix} 10 \\ 0 \\ 4 \end{bmatrix}$$

E teve como resultados obtidos:

$$x[0] = 2.7586206896551726$$

$$x[1] = 2.310344827586207$$

$$x[2] = 1.4137931034482758$$

4. Problema 4.6

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{bmatrix} 1 & 4 & 4 \\ 4 & 7 & 9 \\ 4 & 7 & 18 \end{bmatrix}$$

$$B = \begin{bmatrix} 100 & 100 & 100 \end{bmatrix}$$

E teve como resultados obtidos:

$$x[0] = 3.658536585365854$$

$$x[1] = 12.195121951219512$$

$$x[2] = 0.0$$

Dificuldades enfrentadas:

Não houve dificuldades na implementação desse método.

9. Método de Jacobi

Estratégia de Implementação:

O arquivo contém três funções principais: A função ‘jacobi’, que tem como parâmetros a matriz ‘A’ e a matriz ‘B’ e que realiza o cálculo do método.

A outra função, é a main, nela foi utilizada a biblioteca do Python, Os, necessária para poder ser encontrado o diretório em que o arquivo se encontra e identificar os arquivos ‘in.txt’ e ‘out.txt’, que são os arquivos de entrada e saída respectivamente. Após isso, ele abre o arquivo de entrada e puxa os dados que há nele, e então abre o arquivo de saída e para cada caso teste (foi utilizado um laço for para poder realizar mais de um para cada execução) é feita uma chamada da função ‘jacobi’ para calcular os resultados. Por fim, o resultado final é escrito no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

O arquivo de entrada consiste em N linhas da matriz ‘A’ e uma linha da matriz ‘B’, também é possível por mais de um caso teste por arquivo, então o ‘in.txt’ que foi enviado possui 2 casos testes, o exercício 4.1 e o exercício 4.6 das páginas 150-153 do livro Cálculo Numérico Neide Franco.

O arquivo de saída consiste em N linhas, contendo os resultados do problema teste.

Problema teste:

5. Problema 4.1

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{pmatrix} 8 & -4 & -2 \\ -4 & 6 & -2 \\ -2 & -2 & 10 \end{pmatrix}$$

$$B = \begin{pmatrix} 10 & 0 & 4 \end{pmatrix}$$

E teve como resultados obtidos:

$$x[0] = 2.7585706304205315$$

$$x[1] = 2.3102860122216917$$

$$x[2] = 1.4137634900129648$$

6. Problema 4.6

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{bmatrix} 14 & 4 & 4 \\ 4 & 7 & 9 \\ 4 & 7 & 18 \end{bmatrix}$$

$$B = \begin{bmatrix} 100 & 100 & 100 \end{bmatrix}$$

E teve como resultados obtidos:

$$x[0] = 3.6585369952780264$$

$$x[1] = 12.195122834985114$$

$$x[2] = 4.631218959932287e-07$$

Dificuldades enfrentadas:

No $x[2]$ do exercício 4.6 foi encontrado um valor bem diferente dos outros métodos, porém desconheço o motivo.

10. Método de Gauss-Seidel

Estratégia de Implementação:

O arquivo contém três funções principais: A função ‘metodo_gauss_seidel’, que tem como parâmetros a matriz ‘A’ e a matriz ‘B’ e que realiza o cálculo do método.

A outra função, é a main, nela foi utilizada a biblioteca do Python, Os, necessária para poder ser encontrado o diretório em que o arquivo se encontra e identificar os arquivos ‘in.txt’ e ‘out.txt’, que são os arquivos de entrada e saída respectivamente. Após isso, ele abre o arquivo de entrada e puxa os dados que há nele, e então abre o arquivo de saída e para cada caso teste (foi utilizado um laço for para poder realizar mais de um para cada execução) é feita uma chamada da função ‘metodo_gauss_seidel’ para calcular os resultados. Por fim, o resultado final é escrito no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

O arquivo de entrada consiste em N linhas da matriz ‘A’ e uma linha da matriz ‘B’, também é possível por mais de um caso teste por arquivo, então o ‘in.txt’ que foi enviado possui 2 casos testes, o exercício 4.1 e o exercício 4.6 das páginas 150-153 do livro Cálculo Numérico Neide Franco.

O arquivo de saída consiste em N linhas, contendo os resultados do problema teste.

Problema teste:

7. Problema 4.1

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{bmatrix} 8 & -4 & -2 \\ -4 & 6 & -2 \\ -2 & -2 & 10 \end{bmatrix}$$

$$B = \begin{bmatrix} 10 \\ 0 \\ 4 \end{bmatrix}$$

E teve como resultados obtidos:

$$x[0] = 2.7586199417917965$$

$$x[1] = 2.3103441551260007$$

$$x[2] = 1.4137928193835596$$

8. Problema 4.6

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{bmatrix} 14 & 4 & 4 \\ 4 & 7 & 9 \\ 4 & 7 & 18 \end{bmatrix}$$

$$B = \begin{bmatrix} 100 & 100 & 100 \end{bmatrix}$$

E teve como resultados obtidos:

$$x[0] = 3.6585366322533637$$

$$x[1] = 12.19512192442665$$

$$x[2] = 0.0$$

Dificuldades enfrentadas:

Não houve dificuldades na implementação desse método.

11. Matriz inversa

Estratégia de Implementação:

O arquivo contém três funções principais: A função ‘`resolve_matriz_inversa`’, que tem como parâmetros a matriz ‘A’ e a matriz ‘B’ e que realiza o cálculo do método.

A outra função, é a main, nela foi utilizada a biblioteca do Python, `Os`, necessária para poder ser encontrado o diretório em que o arquivo se encontra e identificar os arquivos ‘`in.txt`’ e ‘`out.txt`’, que são os arquivos de entrada e saída respectivamente. Após isso, ele abre o arquivo de entrada e puxa os dados que há nele, e então abre o arquivo de saída e para cada caso teste (foi utilizado um laço `for` para poder realizar mais de um para cada execução) é feita uma chamada da função ‘`resolve_matriz_inversa`’ para calcular os resultados. Por fim, o resultado final é escrito no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

O arquivo de entrada consiste em N linhas da matriz ‘A’ e uma linha da matriz ‘B’, também é possível por mais de um caso teste por arquivo, então o ‘`in.txt`’ que foi enviado possui 2 casos testes, o exercício 4.1 e o exercício 4.6 das páginas 150-153 do livro Cálculo Numérico Neide Franco.

O arquivo de saída consiste em N linhas, contendo os resultados do problema teste.

Problema teste:

9. Problema 4.1

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{bmatrix} 8 & -4 & -2 \\ -4 & 6 & -2 \\ -2 & -2 & 10 \end{bmatrix}$$

$$B = \begin{bmatrix} 10 & 0 & 4 \end{bmatrix}$$

E teve como resultados obtidos:

$$x[0] = 2.7586206896551726$$

$$x[1] = 2.310344827586207$$

$$x[2] = 1.4137931034482758$$

10.Problema 4.6

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{pmatrix} 14 & 4 & 4 \\ 4 & 7 & 9 \\ 4 & 7 & 18 \end{pmatrix}$$

$$B = \begin{pmatrix} 100 & 100 & 100 \end{pmatrix}$$

E teve como resultados obtidos:

$$x[0] = 3.658536585365853$$

$$x[1] = 12.19512195121951$$

$$x[2] = -2.220446049250313e-16$$

Dificuldades enfrentadas:

No $x[2]$ do exercício 4.6 foi encontrado um valor bem diferente dos outros métodos, porém desconheço o motivo.

12. Condicionamento de matriz

Estratégia de Implementação:

O arquivo contém três funções principais: A função ‘numero_condicao_matriz’, que tem como parâmetro a matriz ‘A’ e que realiza o cálculo do método.

A outra função, é a main, nela foi utilizada a biblioteca do Python, Os, necessária para poder ser encontrado o diretório em que o arquivo se encontra e identificar os arquivos ‘in.txt’ e ‘out.txt’, que são os arquivos de entrada e saída respectivamente. Após isso, ele abre o arquivo de entrada e puxa os dados que há nele, e então abre o arquivo de saída e para cada caso teste (foi utilizado um laço for para poder realizar mais de um para cada execução) é feita uma chamada da função ‘numero_condicao_matriz’ para calcular os resultados. Por fim, o resultado final é escrito no arquivo de saída.

Estrutura dos arquivos de entrada / saída:

O arquivo de entrada consiste em N linhas da matriz ‘A’, também é possível por mais de um caso teste por arquivo, então o ‘in.txt’ que foi enviado possui 2 casos testes, o exercício 4.1 e o exercício 4.6 das páginas 150-153 do livro Cálculo Numérico Neide Franco.

O arquivo de saída consiste em N linhas, contendo os resultados do problema teste.

Problema teste:

11.Problema 4.1

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{bmatrix} 8 & -4 & -2 \\ -4 & 6 & -2 \\ -2 & -2 & 10 \end{bmatrix}$$

E teve como resultado obtido: 5.984781037613702

12.Problema 4.6

O problema fornece um sistema que foi adaptado para as matrizes:

$$A = \begin{pmatrix} 1 & 4 & 4 \\ 4 & 7 & 9 \\ 4 & 7 & 18 \end{pmatrix}$$

E teve como resultado obtido: 9.770978648800124

Dificuldades enfrentadas:

Não houve dificuldades na implementação desse método.

13. Considerações finais

Após concluir este relatório de análise numérica, é evidente a importância dos métodos numéricos na resolução de problemas matemáticos. A implementação de algoritmos e a manipulação de dados permitem alcançar resultados precisos e eficientes. Os testes realizados confirmaram que as implementações utilizadas conseguiram encontrar soluções corretas para os sistemas de equações fornecidos, além de permitir a verificação de outros dados relevantes, como o erro e o tempo de execução.

No entanto, é necessário falar sobre os resultados inconsistentes no $x[2]$ do exercício 4.6, visto que na realização dos métodos de Jacobi e Matriz inversa, seus resultados foram consideravelmente diferentes dos demais métodos implementados.

Portanto, é necessário que futuras implementações sejam desenvolvidas considerando tais inconsistências. É essencial buscar constantemente aprimoramentos nos algoritmos utilizados e incorporar novas funcionalidades para garantir a eficácia dos métodos numéricos empregados. Somente assim será possível resolver problemas cada vez mais complexos e desafiadores com confiabilidade.